

Documentação Técnica

Sistema de Controladoria Municipal — arquitetura, ambiente, banco de dados, segurança, módulos, testes e operação

Guia para quem vai manter ou evoluir o sistema: como subir o ambiente, onde fica cada coisa, as regras de isolamento entre entidades, as integrações públicas e as rotinas de teste e documentação.

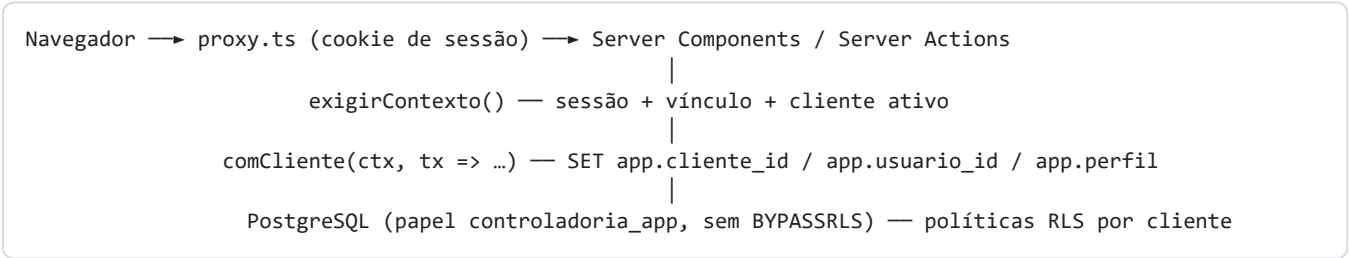
1. Visão geral e stack
2. Ambiente de desenvolvimento
3. Estrutura do projeto
4. Multi-entidade e segurança (RLS)
5. Autenticação e sessão
6. Módulos e rotas
7. Banco de dados e migrações
8. Integrações e alertas fiscais
9. Painel personalizável e preferências
10. Relatórios em PDF e Relatório Anual
11. IA, e-mails e rotinas agendadas
12. Testes
13. Manual, documentação e implantação
14. Convenções de código
15. Histórico da versão 1.1

1

Visão geral e stack

Aplicação web única (Next.js App Router) com PostgreSQL. Cada entidade (Prefeitura, Câmara, autarquia...) é um *cliente* com dados isolados no banco por Row Level Security.

Camada	Tecnologia	Observações
Framework	Next.js 16 (App Router, Turbopack, Server Actions), React 19, TypeScript estrito	Leia <code>node_modules/next/dist/docs/</code> antes de usar APIs novas (AGENTS.md).
Interface	Tailwind CSS 4, componentes shadcn/ui sobre Base UI, lucide-react, sonner	Seis temas em <code>src/lib/temas.ts</code> e <code>globals.css</code> .
Banco	PostgreSQL 17 + pgvector, Prisma 7 (adapter <code>@prisma/adapter-pg</code>)	Cliente gerado em <code>src/generated/prisma</code> .
Autenticação	Sessões próprias no banco, cookie assinado (jose), senhas Argon2 (<code>@node-rs/argon2</code>)	Ver capítulo 5.
Arquivos	S3 compatível (Silo/MinIO) ou disco local	<code>src/lib/armazenamento.ts</code> .
PDF	HTML → PDF com Playwright/Chromium	<code>src/lib/pdf/</code> , <code>src/lib/relatorios/</code> .
IA	OpenAI, Anthropic e Gemini (multiprovedor) com provedor “falso” para testes	<code>src/lib/ia/</code> .
E-mail	Nodemailer (Mailpit em desenvolvimento)	<code>src/lib/email/</code> .
Integrações	IBGE, SICONFI (Tesouro), Portal da Transparência (CGU); links do TCM-BA	<code>src/lib/integracoes/</code> .



2

Ambiente de desenvolvimento

Pré-requisitos

- Node.js 20+ (testado com 24) e npm.
- Docker Desktop (Postgres com pgvector, Redis, Silo/MinIO e Mailpit via `docker-compose.yml`). Sem Docker, `npm run db:local` sobe um Postgres embutido, **mas sem pgvector**: a migração `ia_documentos` falha nele, então use-o só para trabalho que não dependa da IA.
- Edge ou Chrome instalados (ou `npx playwright install chromium`) para PDFs e testes E2E.

Primeira execução

```
cp .env.example .env          # preencha senhas, SESSION_SECRET e CHAVE_CRIPTOGRAFIA
docker compose up -d          # Postgres, Redis, Silo e Mailpit
npm install                    # roda prisma generate no postinstall
npm run db:setup               # cria/atualiza o papel controladoria_app (antes das migrações)
npm run db:deploy              # aplica as migrações
npm run db:seed                # normas, entidades e usuários de exemplo
npm run demo:popular           # (opcional) dados de demonstração de Catolândia/BA
npm run dev                    # http://localhost:3000
```

Porta 5432

Se o Postgres embutido (`db:local`) e o do Docker estiverem ativos ao mesmo tempo, o embutido escuta em `127.0.0.1:5432` e “rouba” as conexões de `localhost`. Pare um deles antes de rodar migrações.

Variáveis de ambiente

Variável	Uso
<code>DATABASE_URL</code>	Dono do banco: só Prisma CLI, setup, seed e scripts de teste.
<code>APP_DATABASE_URL</code>	Papel da aplicação, sujeito a RLS. Nunca use o dono no código da aplicação.
<code>SESSION_SECRET</code>	Assina o cookie de sessão.
<code>CHAVE_CRIPTOGRAFIA</code>	AES-256-GCM dos segredos gravados no banco (chaves de IA).
<code>STORAGE_*</code> , <code>S3_*</code>	Armazenamento de documentos.
<code>SMTP_*</code> , <code>EMAIL_REMETENTE</code> , <code>APP_URL</code>	E-mails de demandas e lembretes.
<code>PORTAL_TRANSPARENCIA_CHAVE</code>	Chave gratuita da API da CGU (vazia = transferências e CEIS/CNEP desabilitados).
<code>IA_PROVEDOR=falso</code>	IA determinística para testes, sem custo.
<code>PDF_CHROMIUM_PATH</code>	Chromium já instalado (opcional).

3

Estrutura do projeto

Pasta	Conteúdo
<code>src/app/(controle)/</code>	Telas da controladoria (perfis Controlador e Equipe), uma pasta por módulo: <code>painel</code> , <code>alertas</code> , <code>demandas</code> , <code>auditorias</code> , <code>autoavaliacao</code> , <code>planos</code> , <code>relatorios</code> , <code>dados-externos</code> ... Cada módulo tem <code>page.tsx</code> , <code>actions.ts</code> (Server Actions) e componentes locais.
<code>src/app/satelite/</code>	Acesso restrito das unidades (demandas e painel da unidade).
<code>src/app/admin/</code>	Administração HorizonAJ: clientes, municípios, usuários, IA, trilha global.
<code>src/app/actions/</code>	Server Actions compartilhadas (sessão, evidências, preferências).
<code>src/components/</code>	<code>ui/</code> (shadcn), <code>shell/</code> (layout, menus por tema), componentes por domínio (<code>alertas/</code> , <code>demandas/</code> , <code>ia/</code> ...) e <code>recolhivel.tsx</code> .
<code>src/lib/</code>	Regras e acesso a dados: <code>auth/</code> , <code>dados/</code> (consultas por módulo), <code>integracoes/</code> , <code>relatorios/</code> , <code>pdf/</code> , <code>ia/</code> , <code>email/</code> , <code>painel/</code> , <code>preferencias*.ts</code> , <code>risco.ts</code> , <code>rotulos.ts</code> , <code>temas.ts</code> .
<code>prisma/</code>	<code>schema.prisma</code> , migrações SQL (com RLS escritas à mão), seeds.
<code>catalogos/</code>	Normas em YAML (OT 05/2024 e Res. TCM-BA 1.120/2005), importadas por <code>npm run catalogos:importar</code> .
<code>scripts/</code>	Setup, seeds de demonstração, testes (<code>testar-*.ts</code> , <code>e2e-*.ts</code>), rotinas (lembretes, sincronização) e geração de documentos.
<code>docs/</code>	Manual do Controlador, esta documentação, implantação e notas de release.

Arquivos compartilhados entre servidor e cliente

Módulos usados por componentes `"use client"` não podem importar nada com `server-only`. Por isso alguns domínios têm dois arquivos: `preferencias-esquema.ts` (puro) e `preferencias.ts` (servidor); `anual-secoes.ts` / `anual-padrao.ts` (puros) e `anual.ts` (servidor).

4

Multi-entidade e segurança (RLS)

- **Tabelas globais** (sem RLS): `municipios`, `clientes`, `usuarios`, `sessoes`, `vinculos_cliente`, `normas`, `requisitos`. Acesse com `db` direto.
- **Tabelas por cliente**: têm `cliente_id`, `ENABLE` + `FORCE ROW LEVEL SECURITY` e a política `isolamento_cliente` (`cliente_id = app_cliente_id()`). Acesse sempre via `comCliente(ctx, tx => ...)`, que abre transação e faz `set_config` de `app_cliente_id`, `app.usuario_id` e `app.perfil`.
- **Satélite**: políticas restritivas (`app_eh_satelite()`, `app_unidades_satelite()`) limitam o perfil às demandas das suas unidades; documentos internos (auditorias, alertas, relatório anual e seu texto padrão) recusam leitura e gravação.
- A aplicação conecta com `controladoria_app` (`NOBYPASSRLS`). `npm run test:rls` verifica o isolamento.

Checklist para uma tabela nova por cliente

1. Modelo no `schema.prisma` com `clienteId` e relação com `Cliente` (`onDelete: Cascade`).
2. Migração com `ENABLE` / `FORCE ROW LEVEL SECURITY`, política `isolamento_cliente` e, se for interna, a política restritiva do satélite; `GRANT` condicional ao papel (bloco `DO $$ ... $$`), como em `20261008120000_painel_alertas_modelo_anual`.
3. CHECKs para JSON (`jsonb_typeof(...) = 'object'`) e faixas.
4. Casos em `testar-rls.ts` ou no teste do módulo (satélite e outro cliente não leem).

Perfis

Perfil	Escopo
<code>CONTROLADOR</code>	Tudo da entidade; decisões finais (concluir ciclo, validar ação, encerrar alerta, aparência, atualizar dados externos).
<code>EQUIPE</code>	Módulos da controladoria sem as decisões finais.
<code>SATELITE</code>	Somente demandas das unidades vinculadas (<code>/satelite</code>).
<code>adminHorizon</code>	Flag do usuário: administração global em <code>/admin</code> .

5

Autenticação e sessão

- `src/proxy.ts` (o antigo *middleware*) só verifica a presença/assinatura do cookie e redireciona para `/login?voltar=...`. A validação real da sessão é feita no servidor, no banco.
- `src/lib/auth/dal.ts`: `obterSessao` (cacheada por requisição), `exigirUsuario`, `obterContexto`, `exigirContexto(perfis)`. Toda página e Server Action começa por um deles.
- Link do município (`/m/<slug>`): a sessão guarda `municipioAcesso` e só alcança as entidades daquele município (admins HorizonAJ ficam livres).
- Tentativas de login são limitadas (`src/lib/limite.ts`) e registradas na trilha global.

6

Módulos e rotas

Os menus vêm de `src/components/shell/menus.ts`; o layout de navegação muda com o tema (`lateral`, `topo`, `trilha`, `faixas`) em `navegacao.tsx` / `app-shell.tsx`.

Rota	Módulo	Dados / regras
<code>/painel</code>	Painel personalizável (capítulo 9)	<code>lib/dados/painel.ts</code> , <code>lib/painel/blocos.ts</code>
<code>/dados-externos</code>	IBGE, SICONFI, Portal, TCM-BA	<code>lib/dados/integracoes.ts</code> , <code>lib/integracoes/</code>
<code>/normas</code> , <code>/autoavaliacao</code>	Normas e ciclos de autoavaliação	<code>lib/dados/autoavaliacao.ts</code> , <code>conformidade.ts</code>
<code>/planos</code>	Planos 5W2H (motor único)	<code>lib/dados/planos.ts</code> , <code>acoes.ts</code>
<code>/ia</code>	Sugestões da IA com revisão humana	<code>lib/ia/</code>
<code>/demandas</code>	Demandas às unidades	<code>lib/dados/demandas.ts</code> , <code>lib/demandas.ts</code>
<code>/auditorias</code>	PAAI, auditorias, checklists, achados	<code>lib/dados/auditorias.ts</code>
<code>/alertas</code>	Alertas (antigo “Medidas”): alertas fiscais + situações (modelo <code>Situacao</code>)	<code>lib/dados/alertas.ts</code> , <code>lib/integracoes/alertas-fiscais.ts</code> . <code>/medidas/*</code> redireciona (308) em <code>next.config.ts</code> .
<code>/documentos</code>	Arquivos e evidências	<code>lib/documentos.ts</code> , <code>lib/armazenamento.ts</code>
<code>/relatorios</code>	PDFs; <code>/relatorios/anual/padrao</code> e <code>/relatorios/anual/[ano]</code>	<code>lib/relatorios/</code> ; rotas em <code>relatorios/pdf/*/route.ts</code>
<code>/unidades</code> , <code>/trilha</code> , <code>/aparencia</code> , <code>/configuracoes</code>	Gestão	<code>lib/auditoria.ts</code> (trilha), <code>lib/temas.ts</code>

Nomes internos × rótulos

O módulo foi renomeado para “Alertas” na interface, nas rotas e nos arquivos; o modelo do banco continua `Situacao` (tabela `situacoes`) e o enum `OrigemPlano.MEDIDA` foi mantido para não exigir migração de dados — o rótulo exibido é “Alerta” (`lib/rotulos.ts`).

7

Banco de dados e migrações

- Migrações em `prisma/migrations/AAAAMDDHHMMSS_nome/migration.sql`; políticas RLS, CHECKs e GRANTs são escritos à mão no mesmo arquivo.
- Desenvolvimento: `npm run db:migrate` (cria) · Produção: `npm run db:deploy` (aplica). Depois de mudar o schema: `npm run db:generate`.
- UUID v7 (`@default(uuid(7))`), datas `timestampz(3)`, nomes de colunas em snake_case via `@map`.

Migração da versão 1.1 — `20261008120000_painel_alertas_modelo_anual`

Objeto	Mudança
<code>usuarios.preferencias</code>	<code>JSONB NOT NULL DEFAULT '{}'</code> + CHECK de objeto. Preferências de exibição por usuário (tabela global, sem RLS).
<code>modelos_relatorio_anual</code>	Nova tabela por cliente (único por <code>cliente_id</code>), <code>secoes JSONB</code> , RLS de isolamento e bloqueio do satélite.

Migração aditiva, sem perda de dados e sem passo manual além do `db:deploy`.

8

Integrações e alertas fiscais

Fonte	O que coleta	Arquivo
IBGE	Município e população (censo/estimativa).	<code>integracoes/ibge.ts</code>
SICONFI	Extrato de entregas; RGF Anexo 1 (pessoal, RCL); RGF Anexo 2 (dívida); RREO Anexos 3 e 6 (RCL, resultado primário).	<code>integracoes/siconfi.ts</code>
Portal da Transparência	Recursos federais por CNPJ, convênios, consulta CEIS/CNEP.	<code>integracoes/portal-transparencia.ts</code>
TCM-BA	Apenas links (sem API pública).	<code>integracoes/tcmba.ts</code>

A coleta roda por cliente (`sincronizar.ts`), grava em `coletas_integracao` e é disparada pelo botão “Atualizar dados” (via `after()`) ou pelo cron `npm run integracoes:sincronizar`.

Dívida e resultado primário para a Câmara (v1.1)

Dívida consolidada e resultado primário são indicadores do ente, publicados pelo Executivo. Para clientes do Legislativo, `coletarSiconfi` agora lê o RGF Anexo 2 (com `co_poder=E`) e o RREO Anexo 6 a partir das entregas da Prefeitura, com o sufixo “— Executivo municipal” na referência. A despesa com pessoal continua sendo a do próprio Poder.

Alertas fiscais (`lib/integracoes/alertas-fiscais.ts`)

Funções puras que transformam a coleta do SICONFI em uma lista ordenada por gravidade (`EXCEDIDO` → `PRUDENCIAL` → `ALERTA` → `INFO`):

- **pessoal**: faixa da LRF ≠ regular (limites em `lrf.ts`: alerta 90%, prudencial 95%, máximo 54%/6%);
- **dívida**: DCL/RCL ≥ 108% (alerta) ou > 120%;
- **entregas**: demonstrativos pendentes (prudencial se mais de dois);
- **coleta**: sem código IBGE, coleta falha ou dados com mais de 40 dias.

Cada alerta traz `registrar`, um link para `/alertas/nova` com título, descrição (com base legal) e probabilidade/impacto sugeridos. A lista é exibida pelo componente `components/alertas/alertas-fiscais.tsx` no Painel, em Alertas e em Dados externos.

9

Painel personalizável e preferências

Catálogo de blocos

`lib/painel/blocos.ts` define os blocos (`id`, título, descrição e página de origem). A ordem do catálogo é a ordem de exibição. Para criar um bloco: adicione-o ao catálogo, ao mapa de ícones/largura em `painel/page.tsx`, ao `switch` de conteúdo e, se precisar de dados novos, a `resumoPainel`.

Preferências (`usuarios.preferencias`)

```
{
  "painel": { "ocultos": ["unidades"], "recolhidos": ["indices"] },
  "dados-externos": { "recolhidos": ["fontes"] },
  "alertas": { "recolhidos": ["painel"] }
}
```

- Esquema e leitura tolerante em `preferencias-esquema.ts` (zod; ids `[a-z0-9-]`, até 50). Guardamos **ocultos** (e não visíveis) para que blocos novos apareçam por padrão.
- `gravarPreferencia()` faz o merge numa única instrução SQL (`preferencias || jsonb_build_object(...)`), sem ler antes: cliques rápidos ou outra aba não sobrescrevem os demais campos.
- Server Actions em `app/actions/preferencias.ts`: `salvarRecolhidos(pagina, ids)` (sem revalidar) e `salvarBlocosPainel(visiveis)` (revalida `/painel`).

Componente `components/recolhive1.tsx`

- `ProvedorRecolhiveis` recebe a página e os recolhidos iniciais (lidos no servidor, sem “pisca”) e grava as mudanças com atraso de 400 ms.
- `BlocoRecolhive1`: cartão com cabeçalho-botão (`aria-expanded`/`aria-controls`), resumo quando recolhido e conteúdo renderizado no servidor (apenas `hidden`).
- `AlternarTodos`: “Recolher tudo / Expandir tudo”.
- Para usar em outra página, inclua o nome em `PAGINAS_PREFERENCIA`.

Menu sem barra de rolagem

O menu lateral foi compactado e as áreas que ainda podem rolar em telas baixas usam a utilidade `sem-barra-rolagem` (`globals.css`, `@utility`); os temas “topo” e “faixas” quebram linha em vez de rolar na horizontal. `npm run test:e2e:painel` verifica os seis temas em 1366×768 e 1366×650.

Relatórios em PDF e Relatório Anual

- Cada relatório tem um montador em `lib/relatorios/<tipo>.ts` que devolve HTML (template `html`` com escape automático), cabeçalho e rodapé; as rotas `relatorios/pdf/*/route.ts` chamam `responderPdf`, que gera o PDF e registra a emissão na trilha.
- O satélite é recusado em `exigirControle()`, além da RLS.

Texto padrão do Relatório Anual (v1.1)

Resolução de cada uma das seis seções (`anual-padrao.ts` → `secoesEfetivas`):

1. texto do exercício (`relatorios_anuais.secoes`);
 2. texto padrão do cliente (`modelos_relatorio_anual.secoes`);
 3. texto-base do sistema (`TEXTO_BASE_ANUAL`).
- Ao salvar um exercício, `somentePersonalizadas()` grava só as seções diferentes do padrão (comparação normalizando quebras de linha e espaços nas pontas). Assim, as demais seguem o padrão inclusive quando ele mudar.
 - Ao salvar o padrão, `somenteDiferentesDoSistema()` não grava seções idênticas ao texto-base.
 - Variáveis `{entidade}`, `{municipio}`, `{uf}`, `{ano}`, `{ano_seguinte}` são substituídas só na emissão (`aplicarVariaveis`); variáveis desconhecidas ficam como estão.
 - Telas: `/relatorios/anual/padrao` (texto padrão + seletor de exercício) e `/relatorios/anual/[ano]`; o formulário `form-relatorio-anual.tsx` serve aos dois modos e mostra, por seção, se segue a referência.
 - Trilha: `relatorio_anual.salvo` (com `secoesPersonalizadas`) e `relatorio_anual.padrao_salvo`.

11

IA, e-mails e rotinas agendadas

- **IA:** configuração por cliente/global em `/admin/ia` (chaves cifradas), mascaramento de dados pessoais antes do envio, citações obrigatórias e revisão humana das sugestões. `IA_PROVEDOR=falso` nos testes.
- **E-mails:** notificações de demandas (`lib/email/`); em desenvolvimento, caixa do Mailpit em `http://localhost:8025`.
- **Cron** (ver `docs/implantacao-vps.md`): `npm run lembretes:prazo` (lembretes de prazo) e `npm run integracoes:sincronizar` (dados externos, diário).

12

Testes

Os testes são scripts `tsx` com verificações explícitas (saída “OK/FALHA” e código de saída $\neq 0$ em falha). Os que importam módulos `server-only` rodam com `--conditions=react-server` (já configurado no `package.json`). Requerem migrações + seed; os E2E requerem também `npm run dev`.

Comando	Cobre
<code>npm run typecheck</code> · <code>npm run lint</code>	TypeScript estrito e ESLint.
<code>test:rls</code>	Isolamento entre clientes e restrições do satélite.
<code>test:acesso-municipio</code> , <code>test:senha</code>	Link do município, sessão, política de senha.
<code>test:autoavaliacao</code> , <code>test:risco</code> , <code>test:auditorias</code>	Regras de conformidade, matriz de risco, ciclo da auditoria.
<code>test:relatorios</code>	Todos os PDFs, regras do texto padrão do relatório anual e RLS dos textos.
<code>test:preferencias</code>	Leitura tolerante, merge atômico e CHECK das preferências.
<code>test:integracoes</code> (<code>-- --real</code>)	Parsing das APIs, prazos do SICONFI, limites da LRF, dívida para a Câmara, alertas fiscais; opcionalmente as APIs reais.
<code>test:ia</code> , <code>test:config-ia</code> , <code>test:emails</code>	IA com provedor falso, configuração e modelos de e-mail.
<code>test:e2e</code> , <code>test:e2e:demandas</code>	Fluxos principais no navegador (seed de exemplo).
<code>test:e2e:painel</code>	Painel personalizável, persistência dos blocos recolhidos, Alertas, redirecionamento de <code>/medidas</code> , texto padrão e menu sem barra de rolagem nos seis temas (dados de Catolândia).

Dados de teste

Os testes que dependem do seed escolhem os clientes mais antigos (`ORDER BY criado_em`), porque o banco pode conter também os dados de demonstração. Os testes que alteram registros (tema, preferências, texto padrão) restauram o estado original no `finally`.

13

Manual, documentação e implantação

Comando	Gera
<code>npm run manual:pdf -- --capturar</code>	Refaz as capturas (<code>scripts/manual/capturas.ts</code> , 1366×768, dados de Catolândia) e gera <code>docs/manual/Manual-do-Controlador.pdf</code> a partir de <code>manual-controlador.html</code> , com o sumário paginado automaticamente.
<code>npm run docs:tecnico</code>	Este documento: <code>docs/tecnico/Documentacao-Tecnica.pdf</code> a partir de <code>documentacao-tecnica.html</code> .
<code>docs/notas-de-release/</code>	Notas de release por versão (Markdown).
<code>docs/implantacao-vps.md</code>	Implantação com Docker Compose, proxy reverso (Caddy/OpenResty) e cron.

Publicar a versão 1.1

```
git pull
docker compose -f docker-compose.prod.yml build app
docker compose -f docker-compose.prod.yml run --rm app npm run db:deploy
docker compose -f docker-compose.prod.yml up -d app
```

Convenções de código

- Código, rotas, tabelas e mensagens em português; nomes descritivos (`exigirContexto`, `calcularAlertasFiscais`).
- Páginas são Server Components; interatividade em componentes `"use client"` pequenos. Mutação por Server Action que começa com `exigirContexto`/`exigirUsuario`, valida com zod, grava via `comCliente`, registra na trilha e chama `revalidatePath`.
- Retorno padrão das actions: `EstadoAcao` (`{ ok, erro, mensagem }`); mensagens de erro amigáveis via `mensagemDeErro`.
- Estados vazios, de carregamento e de erro explícitos nas telas; acessibilidade (rótulos, `aria-*`, foco visível).
- Rótulos exibidos ficam em `lib/rotulos.ts`; regras puras em módulos sem dependência de servidor, para serem testadas sem banco.
- Formatação: Prettier com largura de linha 150 (padrão dos arquivos existentes).

15

Histórico da versão 1.1 (08/10/2026)

Mudança	Arquivos principais
Medidas → Alertas (rótulos, rotas, PDF, trilha) com redirecionamento	<code>app/(controle)/alertas/</code> , <code>lib/dados/alertas.ts</code> , <code>lib/relatorios/alertas.ts</code> , <code>components/alertas/</code> , <code>next.config.ts</code>
Alertas fiscais automáticos (folha, dívida, SICONFI)	<code>lib/integracoes/alertas-fiscais.ts</code> , <code>components/alertas/alertas-fiscais.tsx</code>
Dívida/resultado primário também para a Câmara	<code>lib/integracoes/siconfi.ts</code> , <code>dados-externos/page.tsx</code>
Painel personalizável com índices em destaque	<code>app/(controle)/painel/</code> , <code>lib/painel/blocos.ts</code> , <code>lib/dados/painel.ts</code>
Blocos recolhíveis lembrados por usuário	<code>components/recolhivel.tsx</code> , <code>lib/preferencias*.ts</code> , <code>app/actions/preferencias.ts</code>
Menu sem barra de rolagem em todos os temas	<code>shell/navegacao.tsx</code> , <code>shell/app-shell.tsx</code> , <code>globals.css</code>
Texto padrão do Relatório Anual	<code>lib/relatorios/anual-padrao.ts</code> , <code>relatorios/anual/</code> , <code>lib/relatorios/anual.ts</code>
Testes	<code>testar-preferencias.ts</code> , <code>e2e-painel.ts</code> , casos novos em <code>testar-integracoes.ts</code> e <code>testar-relatorios.ts</code> ; correções de espera em <code>e2e-smoke.ts</code>